University of Science VNU-HCM

Faculty of Information Technology
Advanced Program in Computer Science

???

Report

Students:
Tran Canh Anh Tuan 24125048
Le Dat Phuc An 24125052
Vong Chi Van 24125041
Pham Nguyen Minh Quan 24125041

Lecturers:
Nguyen Ngoc Toan, MSc.
Nguyen Ngoc Minh Chau, MSc.
Le Thi Nhan, PhD

2026

Summary

§1 Introduction

§1.1 Project Background

The School of Computer Science manages shared physical spaces (classrooms, laboratories, meeting rooms, auditoriums) for teaching, seminars, examinations, workshops, and student activities. Space requests are currently handled manually via email, phone, and spreadsheets, leading to booking conflicts, use of spaces under maintenance, and loss of usage history. This project develops a relational database system to replace the manual process.

§1.2 Project Objectives

- Centralize the management of all bookable campus spaces and their facilities.
- Prevent conflicting bookings and block reservations for unavailable spaces.
- Track maintenance issues from report through resolution.
- Preserve complete historical records of bookings and maintenance.
- Support operational decision-making with booking and usage data.

§1.3 Project Deliverables

Deliverable	Description
Business Requirement Analysis	Requirement extraction and analysis
ER Diagram	Conceptual database design
Logical Schema	Relational database design
SQL Implementation	Database implementation
Sample Data	Testing dataset
SQL Queries	Business queries

§2 Database Design

§2.1 Business Requirement Analysis

§2.1.1 Business Purpose

The system manages the booking and usage of shared campus spaces for the School of Computer Science, replacing a manual spreadsheet/email process with a structured database that handles booking requests, approvals, check-in/check-out, maintenance tracking, and historical reporting.

§2.1.2 Actors

Role	Description
Student	Submits booking requests for student activities; views own bookings
Lecturer	Submits booking requests for lectures and examinations
Teaching Assistant	Submits booking requests for workshops and tutorials
Facility Staff	Approves/rejects bookings, performs check-in/check-out, manages maintenance
Department Administrator	Views departmental booking history
Facility Manager	Oversees all spaces, bookings, maintenance, and facility statuses

§2.1.3 Core Entities

Entity	Key Attributes
`USER`	`user_id` (PK), `full_name`, `email`, `phone`, `role`, `department`, `account_status`
`SPACE`	`space_code` (PK), `space_name`, `space_type`, `building`, `floor`, `room_number`, `capacity`, `current_status`, `usage_policy`
`FACILITY`	`facility_id` (PK), `space_code` (FK), `facility_name`, `description`
`BOOKING_REQUEST`	`booking_id` (PK), `user_id` (FK), `space_code` (FK), `requested_start_time`, `requested_end_time`, `purpose`, `expected_participants`, `booking_type`, `status`
`BOOKING_APPROVAL`	`approval_id` (PK), `booking_id` (FK, UNIQUE), `decided_by_user_id` (FK), `decision_time`, `decision_note`, `rejection_reason`
`USAGE_SESSION`	`session_id` (PK), `booking_id` (FK, UNIQUE), `actual_start_time`, `actual_end_time`, `checked_in_by_user_id` (FK), `completed_by_user_id` (FK), `initial_condition`, `final_condition`, `usage_notes`
`MAINTENANCE_RECORD`	`maintenance_id` (PK), `space_code` (FK), `reporter_user_id` (FK), `assigned_staff_user_id` (FK), `problem_description`, `start_time`, `completion_time`, `status`, `result_note`

§2.1.4 Business Rules

1. No overlapping approved bookings for the same space.
2. Spaces with status `under_maintenance`, `temporarily_closed`, or `retired` cannot be booked.
3. Spaces with active maintenance records cannot be booked.
4. `requested_end_time` must be greater than `requested_start_time`.

5. BOOKING_APPROVAL is optional, at most one per BOOKING_REQUEST (1:0..1).
6. USAGE_SESSION is optional, at most one per BOOKING_REQUEST (1:0..1).
7. Rejected bookings must preserve rejection_reason.
8. Check-in must record actual_start_time, checked_in_by_user_id, initial_condition.
9. Completion must record actual_end_time, completed_by_user_id, final_condition, usage_notes.
10. Historical data must be preserved (no hard deletes).

§2.1.5 Requirement Summary

Requirement	Database Support
User management	USER with role and account status
Space inventory	SPACE with type, capacity, and status
Facility tracking	FACILITY linked to SPACE via FK
Booking workflow	BOOKING_REQUEST with status lifecycle
Approval management	BOOKING_APPROVAL (1:0..1 to request)
Usage tracking	USAGE_SESSION for check-in/check-out
Maintenance management	MAINTENANCE_RECORD from report to resolution
Conflict prevention	Triggers/application logic on approved bookings
History preservation	Soft-delete policy across all relations

§2.2 Conceptual Database Design

§2.2.1 Entity Relationship Diagram

The ERD was developed using Mermaid notation, modeling seven entities with their attributes, relationships, cardinalities, and participation constraints. The complete diagram is provided in the project deliverables (output/02-erd-design-G08.md).

§2.2.2 Design Decisions

- **Entity identification:** Seven entities map one-to-one to distinct real-world concepts—no merging or over-splitting.
- **Relationship design:** USER participates in multiple relationships with distinct semantic roles (requester, approver, check-in staff, reporter, assigned staff), modeled via separate foreign keys.
- **Cardinalities:** BOOKING_APPROVAL and USAGE_SESSION use 1:0..1 (not 1:N), reflecting at-most-one approval/session per booking.
- **Participation:** Total participation for dependent entities (e.g., every FACILITY must belong to a SPACE); partial participation where entities may exist independently.

§2.2.3 Relationship Summary

Relationship	Cardinality
Space contains Facility	1 : N
User submits Booking_Request	1 : N
Space receives Booking_Request	1 : N
Booking_Request has Booking_Approval	1 : 0..1
User performs Booking_Approval	1 : N
Booking_Request creates Usage_Session	1 : 0..1
User checks in Usage_Session	1 : N
User completes Usage_Session	1 : N
Space has Maintenance_Record	1 : N
User reports Maintenance_Record	1 : N
User assigned to Maintenance_Record	1 : N

§2.3 Logical Database Design

§2.3.1 Relational Schema

The ERD was converted into seven relations. The full schema definitions with data types are provided in the project deliverables (`output/03-logical-design-G08.md`).

§2.3.2 Relation Summary

Relation	PK	Foreign Keys	Candidate Keys
USERS	user_id	—	email
SPACES	space_code	—	(building, room_number)
FACILITY	facility_id	space_code	—
BOOKING_REQUEST	booking_id	user_id, space_code	—
BOOKING_APPROVAL	approval_id	booking_id (UNIQUE), decided_by_user_id	booking_id
USAGE_SESSION	session_id	booking_id (UNIQUE), checked_in_by_user_id, completed_by_user_id	booking_id
MAINTENANCE_RECORD	maintenance_id	space_code, reporter_user_id, assigned_staff_user_id	—

All eleven foreign keys implement the ERD relationships. The UNIQUE constraint on `booking_id` in `BOOKING_APPROVAL` and `USAGE_SESSION` enforces the 1:0..1 cardinality.

§2.3.3 Normalization

All relations satisfy Third Normal Form (3NF):

- **1NF:** All attributes are atomic.
- **2NF:** No partial dependencies (all PKs are single-attribute).
- **3NF:** No transitive dependencies among non-key attributes.

§2.4 Database Validation

§2.4.1 Requirement Traceability

Every business requirement maps to a database component: user management to `USERS`, space inventory to `SPACES`, facility tracking to `FACILITY`, booking workflow to `BOOKING_REQUEST`, approval decisions to `BOOKING_APPROVAL`, usage tracking to `USAGE_SESSION`, and maintenance management to `MAINTENANCE_RECORD`.

§2.4.2 Constraint Analysis

Business Rule	Schema Constraint	Additional Enforcement
No overlapping bookings	—	Trigger / application logic
Unavailable spaces not bookable	CHECK on `current_status`	Application status check
Active maintenance blocks booking	—	Application logic
End time > start time	CHECK constraint	—
At most one approval per booking	UNIQUE on `booking_id`	—
At most one session per booking	UNIQUE on `booking_id`	—
Rejection reason preserved	—	Application logic
Valid user roles	CHECK on `role`	—
No hard deletes	—	Application policy

§2.4.3 Design Limitations

The following constraints cannot be expressed as single-table CHECK constraints and require triggers or application logic:

- **Overlapping booking prevention:** Requires querying existing approved bookings for time-range overlap detection.
- **Maintenance-based booking blocks:** Requires cross-table join between `BOOKING_REQUEST` and `MAINTENANCE_RECORD`.
- **Conditional NOT NULL:** Fields like `rejection_reason` and completion fields are conditionally required based on booking status.